From parsing to a bounded buy

Weekly synthesis: 12-18 October 2026 | Written 18 October | Synthetic evidence only

Sidiq review status: Not yet reviewed. Integrated 22 October 2026.

Sections below through Decisions and input describe the October 18 observation, including its then-pending work and publication state. The final integration section records subsequent progress.

What changed this week

C003a's parser teaching slice is complete. The accounting sequence now has published valuation, buy-and-fee equations, an account struct and a bounded validated buy. Today's independent boundary study is complete locally and pending integration. L001 reproduction and L002 decision/execution timing remain unfinished. This is progress in accounting mechanics, not a completed backtester.

The baseline is last Sunday's 0a51f14; current main is 0cbaa0246cc4ffa17fc5f7072d156d5b384af40d. Eight actual commits follow that baseline. Two publish previous-week material, five publish teaching slices, and one repairs a documentation insertion. Publication dates do not replace study dates.

Published	Commit	Actual contribution
Oct 12	609225c	Oct 11 parser reproduction
Oct 12	c8c925e	Oct 11 weekly synthesis and paced backlog
Oct 13	3b26d43	Oct 12 parser consolidation/readiness
Oct 13	fc26041	Cash and marked wealth intuition
Oct 14	5405c49	One-buy fee equations
Oct 14	73fa0f5	Remove stray command from historical README paragraph
Oct 17	ce2ce13	Oct 15 account struct
Oct 18	0cbaa02	Oct 17 validated buy

Study and publication are separate

Six new study packages exist for October 12, 13, 14, 15, 17 and 18. No October 16 lesson package or persisted draw was found; the reason is unknown. It must not be called a zero-draw study day. October 15's draw was zero, with its lesson preserved and integrated October 17. Morning L001e is not repeated by this synthesis and is not yet part of tracked Make checks or remote CI.

London-date draws for Oct 12-18 are 2, 3, 3, 0, unavailable, 1, 1. Actual commit counts are 2, 2, 2, 0, 0, 1, 1, using committer timestamps converted to Europe/London. The October 14 correction counts against that day's allowance; it is not another lesson. No missing draw is created retrospectively. Today: draw 1, actual count 1, weekly new commits 0, remaining slots 0.

This report, PDF and backlog proposal stay under work/pending/2026-10-18/weekly. No tracked file is changed. Fetch and independent remote lookup match local main; no unpushed commit needs a retry. No weekly push is needed or attempted. The morning publication is independently verified.

Concepts worth retaining

Parsing and calculation have different contracts. October 12's eight traces produce four parser rejections, three finite balances and one accepted horizon whose positive-rate calculation overflows. The maximum tested int, 2147483647, is valid whole-year text, but 1000*1.05^n cannot produce a finite double result there. At zero rate the same accepted horizon returns 1000. Rejecting all large year counts would change the parser policy, rather than fix model arithmetic.

An account snapshot contains cash C and shares q. A mark m is a valuation input in money per share. Holdings H=q*m and wealth W=C+H have money units. With C=800 and q=4, marks 50, 55 and 45 produce wealth 1000, 1020 and 980, while cash stays 800. A higher mark is not a cash deposit. The struct groups the two owned quantities; it does not itself enforce financial validity.

A buy moves value and charges a fee

For quantity n, fill price p and fee f: debit D=n*p+f; new cash C'=C-D; new shares q'=q+n. With C=800, q=4, n=2, p=50 and f=3, debit is 103, new cash 697 and new shares 6. At the unchanged mark m=50, wealth falls from 1000 to 997. The purchase notional moves from cash into holdings; the fee accounts for the loss. Cash plus notional plus fee reconciles to 800.

The more general same-mark identity is W'-W=n*(m-p)-f. Only when m=p does it simplify to -f. This is an algebraic deduction from the ledger, not a measured spread or a new experiment. Execution away from the mark can therefore change wealth beyond the explicit fee. Real spread, slippage, market impact, financing, tax and settlement remain unmodelled.

Starting state	Request	Expected final state
Cash 800, shares 4	Buy 2 at 50, fee 3	Accept: cash 697, shares 6
Cash 102, shares 4	Buy 2 at 50, fee 3	Reject: cash 102, shares 4
Cash 103, shares 4	Buy 2 at 50, fee 3	Accept: cash 0, shares 6

Why validation comes before mutation

try_buy takes AccountState by reference, so writing its fields changes the caller's state. It returns a Boolean acceptance flag. Cash, starting shares and fee lie in 0..10000; quantity and price in 1..10000; final shares cannot exceed 10000. These are educational whole-unit caps. The function checks every rule before changing either field. Rejection does not charge a fee.

After reserving the fee in a local value, it checks n <= (C-f)/p and n <= 10000-q before multiplication/addition. Positive p makes division safe; accepted debit cannot exceed C, which is at most 10000. This bounds the arithmetic in the stated domain. An int parameter still cannot recover discarded fractional text; C003b and a genuine external-input policy remain work.

Important results and negative evidence

Published L001d checks three accepted and eleven rejected unchanged states. Today's pending L001e instead compares a direct-product Python integer ledger with the C++ division/capacity guards. Six hand anchors constrain the oracle. A 7776-tuple edge grid and 575 extra unique threshold neighbors give 8351 comparisons: 250 accepted and 8101 rejected unchanged. Each case starts from fresh state. Two fresh sanitizer builds, each run twice, reproduced the morning result. Those are archived morning observations, hash-checked here, not another weekly experiment run.

The selected edge grid is exhaustive only over its six-value lists. These counts are not probabilities, confidence levels or a coverage percentage. Agreement supports the written contract, which both implementations share; it does not establish that the contract models a broker.

A surviving mutation changed the interpretation

The morning control changed price_units <= 0 to price_units == 0. The runner initially expected detection, but the mutant passed. With nonnegative cash remaining and a negative price, integer division gives a nonpositive quotient. Positive quantity then fails the later affordability guard. Thus removing that early rejection did not produce an accepted negative-price trade in the test.

The failed expectation was investigated and preserved. The original positive-price guard remains. A surviving mutant can reflect redundant behavior, an untested distinction or a defective oracle; it is a question to explain, not permission to claim a perfect mutation score. Here the source's later rejection explains the observation. No all-program equivalence proof is claimed.

Cash and share-cap exclusive-equality mutants were detected at their intended thresholds. Earlier controls also caught share-plus-price arithmetic, omitted fees, missing acquired shares, swapped struct fields and charging a fee on rejection. They establish that specific mistakes are observable; warnings alone did not catch these financially wrong but compilable programs.

Failures and gaps retained in the record

October 14's overly broad README command insertion required a separate corrective commit. October 13 integration formatting and October 18's long scope string initially failed quality checks, then passed after correction. PDF spill pages and font-cache issues were repaired and rechecked during the daily runs; an optional contact-sheet helper lacked Pillow, so original pages were inspected directly. These are recorded workflow failures, not hidden research wins.

No October 16 output was located. October 17 publication needed a permitted retry after an approval timeout. Today's weekly restricted fetch failed DNS; approved fetch succeeded. No new approval rejection or bypass occurred. No missing lesson, personal review or coverage is inferred.

L001e does not test sequences of orders, delayed execution or future-price invariance. No sells, partial fills, debt, shorting, fractional money, multi-asset identities or calibrated costs exist in this ledger. There is no historical market sample, investment return, alpha, speedup or production-readiness result. The programme's older empirical access gates are still unresolved.

Code-quality evidence observed this afternoon

make check passes on unchanged main: 608 Python tests, Ruff lint, 57 formatted files and mypy on 21 package source files. Python test count remains 608; new C++ lessons did not add Python unit tests to that count. mypy's package scope does not imply every verification script is typed. pip --no-cache-dir check reports no broken requirements. No dependency was added this week.

The C++ checks pass 66 savings and 30 parser cases, 75 parser-oracle comparisons, four exact golden outputs, correct/wrong fractional controls, eight readiness traces, marked-wealth, buy-equation, struct and 14-case validated-buy checks. Make reuses standard binaries this afternoon. The readiness, struct and buy verifiers separately compile fresh temporary drivers; the buy driver enables UndefinedBehaviorSanitizer. Do not describe this as a full weekly rebuild.

Evidence	What it supports	What it does not establish
Exact output, status and stderr checks	Repeatable fixture behavior	Every possible input or platform
Independent arithmetic and hand anchors	Agreement across calculation methods	Independent research authors or valid market assumptions
Deliberate controls and -O refusal	Some checker failures are observable	Complete mutation or branch coverage
Sanitizer runs without diagnostics	No detected issue on executed paths	Proof of absence of undefined behavior
Manifest hashes and receipts	Original bytes and study/publication provenance	Truth of every scientific claim

No line or branch coverage percentage was measured. No new coverage tool is installed merely to provide a number. The morning 8351-case verifier remains pending outside tracked make check, Ruff's default paths and remote CI; its separate checks are documented in its own evidence. Its ten inputs and seven artifacts are verified unchanged against the integration manifest here.

Repository and CI status

Read-only GitHub inspection finds zero open issues. Workflow run 39110153739 completed successfully for exact main 0cbaa0246cc4ffa17fc5f7072d156d5b384af40d. The workflow declares Ubuntu jobs for Python 3.11, 3.12 and 3.13, make check, the availability experiment and the original note renderer. Its successful result is not evidence that pending L001e or this weekly PDF ran in remote CI.

https://github.com/SidiqB/quant-research-apprenticeship/actions/runs/39110153739

Weekly qa/make-check.log records the current run. The integration manifest pins source evidence and deliverables. The companion validation note records complete PDF text comparison, page inspection, content scans and backlog patch checks. Personal review remains Not yet reviewed; repository checks and standing publication authority cannot establish Sidiq's understanding.

Next priorities from evidence

October 19's trend implementation gate is not met. L001e is already studied locally, so the next learning step is L001f reproduction/consolidation, not another boundary lesson. The proposed backlog replaces Oct 19-25's executable trend slots with one bounded prerequisite per day. It preserves the original trend window as deferred intent and does not compress later projects.

Earliest date	Bounded step	Acceptance evidence before advancing
Oct 19	L001f reproduce and consolidate	Fresh reproduction; reconcile fee, wealth and rejected unchanged state
Oct 20	L002a information versus execution	Hand timeline: what is known at decision; later fill may differ
Oct 21	L002b delayed-event contract	Define known fields, event order, fee and missing-fill policy
Oct 22	L002c one delayed fixed order	Small C++ trace; introduce only required syntax before use
Oct 23	L002d future-price mutation	Earlier decision unchanged; later fill/wealth may change; deliberate leaking control
Oct 24	L002e timing failure and costs	One adverse-fill hand scenario; distinguish decision from execution
Oct 25	L002f reproduction/readiness	Reproduce bounded evidence; weekly synthesis remains separate

Dates are conditional earliest slots, not completion promises. Repeat prerequisites where unclear; split again if a step exceeds 45-90 minutes. L002a/b require no vectors, full CLI or generic event engine. L002c must remain one fixed order: do not add headers, enums, containers and scheduling together. This work prepares timing reasoning; it is not a trend signal or live execution model.

C004 returns/vectors follows timing readiness before rolling-window trend work. C005 statistics precedes volatility scaling. Full C003b CLI remains separate; a fixed fixture need not wait for it. Trend, cross-sectional ranking/ties (Q011), forward targets (Q012) and benchmark (Q013) retain their specific prerequisites. The Oct 26 pairs window is conditional too: do not jump past the deferred trend/return foundations because a calendar date arrives. Reassess at next synthesis.

Integration queue and data gates

On a later positive-budget date, first verify both Oct 18 manifests and count actual commits. L001e is one meaningful ready unit; this synthesis/backlog is another. Publish only within the remaining allowance, with current validation and original study dates. Merge backlog changes against then-current progress; never replace a newer file with this saved snapshot. Reconcile README, roadmap, C++ plan and changelog when actually integrating. Do not create extra lessons.

Q004a1 entitlement/output rights precedes Q004a2 licensed sample audit. Q005b1 source/knowledge contract and Q005b2 two-batch adapter retain provenance, identity, vintage, terminal-proceeds and return-basis requirements, with Q013 still a dependency. Synthetic continuation needs no new data access; empirical work does. No narrower historical-vintage claim is silently accepted.

Employer-skill connections

The September 21 career study samples 49 postings across eight employers and mixed seniority. Its mention counts are neither current vacancy counts nor evidence of meeting requirements.

AQR's recorded validation, Git, PyTest and CI/CD requirements connect to the ledger checks. Jane Street's recorded reproducible infrastructure connects to temporary builds and manifests. These are curriculum inferences from saved evidence; requirements were not refreshed today.

AQR recorded role: https://careers.aqr.com/jobs?gh_jid=8088184&gh_jid=8088184

Jane Street recorded role: https://www.janestreet.com/join-jane-street/apply/7437779002?gh_jid=7437779002

Interview story: reconcile cash/holdings, preserve rejected state, use different oracle arithmetic and explain a surviving mutation. Statistics, market calibration and performance remain gaps.

Sources and evidence trail

Primary language/tool references checked October 18: Microsoft Multiplicative Operators explains integer division; Clang documents sanitizer instrumentation and execution checks. The ledger domain, oracle, mutation explanation and scheduling decisions are this project's reasoning.

https://learn.microsoft.com/en-us/cpp/cpp/multiplicative-operators-and-the-modulus-operator?view=msvc-170

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Local evidence: October 12/13/14/15/17 research_log notes and milestone validations; years_parser and portfolio_accounting experiments; October 18 pending note/result/manifest; and career_research/role_skill_analysis.md. The integration manifest pins the exact source paths.

Revision questions and answers

1. Why can a valid year count fail later? Parsing validates spelling/range; model growth can still exceed finite arithmetic. Valid zero is not a fallback for invalid input.

2. Does a higher mark increase cash? No. It changes q*m; cash changes only through a specified cash flow. A mark is not a guaranteed executable quote.

3. When does buying reduce wealth by exactly the fee? When the mark is unchanged and equals the fill, with no other costs/cash flows. Otherwise n*(m-p)-f is the same-mark change.

4. What must rejection preserve? Both original fields, even for invalid starting state. A false flag alone cannot detect an incorrectly charged fee.

5. Do 8351 passing cases prove timing correctness? No. They reset state per tuple and contain no decision/execution timeline. L002's future-price test is still required.

Decisions and input for Sidiq

The concrete pacing change is another prerequisite week before strategy implementation, allowed by the curriculum's repeat-and-simplify rule. Would more repetition of accounting or references help? No answer is required for the next bounded lesson. Existing licensed access or a source/budget preference is needed before acquisition. The separate finance-project identity remains unresolved.

Sunday synthesis remains 16:00 Europe/London, including DST: Oct 25 is 16:00 UTC, following Oct 18's 15:00 UTC slot. Stop new programme work after Dec 31 and request the next scope.

Integration record - 22 October 2026

The October 18 report is integrated with its original observation date preserved. All 67 inputs, five artifacts and five QA evidence records match either their unchanged local bytes or, for eight subsequently changed tracked inputs, their exact original baseline Git blobs. The changes are later progress documents, Makefile checks and the accounting project README; no source evidence is silently replaced. Original pending files remain intact. The saved backlog proposal is merged selectively with the newer completed L001e/f records, never copied over current progress.

Since the original synthesis, L001e was integrated October 19 and L001f October 20. L002a's hand timeline was studied October 20 and remains locally pending under the commit cadence. No October 21 lesson package was found; neither completion nor a reason is inferred. The next bounded learning step is L002b's event ordering, fee timing and missing-fill contract, followed by the small C++ component. The original earliest dates are historical targets, not evidence that later lessons occurred. Trend, returns/statistics and data-access prerequisites remain.

Current make check passes 608 Python tests, Ruff lint and 59 formatted files, mypy on 21 package source files, and all C++ checks including the integrated boundary and fresh-accounting reproduction checks. The reproduction builds five temporary C++ programs and compares all five archived results; buy checks use UndefinedBehaviorSanitizer. Existing standard Make binaries are reused on this documentation integration. Dependency consistency passes; no dependency or production code is changed. The known negative-price mutation survivor remains a limitation.

The integrated PDF is regenerated and checked for complete source-text agreement, page bounds and every-page visual layout. The companion integration validation identifies current evidence separately from the original October 18 validation. No line/branch coverage or market claim is added.

Today's shared London-date draw is one, with zero earlier commits. This coherent report/backlog integration uses that allowance; today's new learning must stay in ignored pending storage. At pre-publication inspection local main b50d8a7 was one commit ahead of remote 2b6bc10, whose CI run 39246361734 succeeded; zero open issues were found. The prior push rejection is historical. Both the pending existing commit and this validated integration are authorised for normal publication, subject to current platform checks. Final push status belongs in the run receipt and briefing, rather than being predicted in this report.

Daily work remains scheduled at 08:00 Europe/London including DST; October 25's scheduled 08:00 local time is 08:00 UTC. Sunday synthesis remains 16:00 local. Stop new research after December 31 and request the next scope. No input blocks synthetic work; licence/access evidence is needed before real data acquisition. Personal review remains Not yet reviewed.